Testbericht 1: Kernel, Sprache, Engine, CLI

controller-cad · Stand 2026-10-23 01:24 · Commit fd8dda6 2026-10-23 01:24 Fassade: bei durchgefallener Prüfung SMLibs eigenes Ergebnis behalten (Branch `worktree-agent-a45db6d1ee5be50fc`)

202	33/33	1,58 MiB	141 ms
Rust-Tests bestanden (0 Fehler, 0 ignoriert)	OpenSCAD-Modelle innerhalb der Toleranz	Wasm gesamt, Brotli (SMLib 1,24 MiB, Engine 0,34 MiB)	Quelltext → Wurzel-Mesh (Beispiel gehaeuse, Node, kalt)

1 Umfang

Geprüft werden die vier unteren Schichten der Anwendung (UI nicht Teil dieses Berichts):

- **Kernel:** SMLib/SM_API (NVIDIA usd-brep @ c9c5979) hinter einer C++-Fassade mit flacher C-ABI (Handles, Fehlercodes, Flächen-Tags für die Provenienz), nativ gelinkt und als Emscripten-Wasm (`smlib.wasm`); Rust-Anbindung `kernel-smlib`. Operationen: Primitive, Transformationen, Booleans, Fillet/Fase, Extrusion (mit twist/scale), Rotation, Loft, Sweep, NURBS-Fläche, thicken, shell, Kanten/Flächen-Abfragen, Klassifizieren, Tessellierung, Masseeigenschaften.
- **Sprache:** OpenSCAD-Dialekt (`cad-syntax`, `cad-eval`): Variablen, Funktionen, Module, `children()`, `for` / `if` / `let` / `each`, Listenausdrücke, `echo` / `assert`, Modifier, 2D/3D-Primitive, Extrusionen; dazu die Erweiterungen der Spec: Raster (`grid`), Anker, benannte Instanzen, `box()` zwischen Rasterpunkten, Fillets/Fasen mit Zonen, Loft, Sweep, Griffe.
- **Engine:** `cad-engine` wertet den Geometriebaum schrittweise mit Cache (inhaltsbasierte Schlüssel) aus, ordnet Flächen und Kanten Zeilen zu, wählt Kanten über Zonen und exportiert STL/3MF; `cad-session` stellt das Protokoll für Worker und CLI.
- **CLI:** `cad check | eval | inspect | render | export` mit `--json`; `render` nutzt den Software-Rasterizer `cad-render`.

2 Testumgebung

System	Ubuntu 24.04.5 LTS (Linux 6.18.44-fc-v140), Intel(R) Xeon(R) Processor @ 2.80GHz, 4 Kerne, 16 GiB RAM, Container ohne GPU
Rust	rustc 1.97.0 (2d8144b78 2026-07-07); cargo 1.97.0 (c980f4866 2026-06-30)
Kernel	usd-brep @ c9c5979 (SMLib, SM_API), native vorgebaut: /home/user/controller-cad/smlib/build-native
Emscripten	emcc (Emscripten gcc/clang-like replacement + linker emulating GNU ld) 6.0.13 (e13ee988ce3d87a738dd8f3661d02223308c412a)
wasm-bindgen	wasm-bindgen 0.2.129
Node	v22.22.0
OpenSCAD (Referenz)	OpenSCAD version 2021.01; Bilder mit xvfb-run, orthografisch
PDF	Chromium aus Playwright 1.56.1
Python	3.13.16

Release-Build der CLI: 0,1 s (inkrementell). Alle Zahlen stammen aus einem Lauf von `tools/report/report1.py`; Zeiten sind Wanduhrzeiten auf dieser Maschine.

3 Testergebnisse

3.1 Rust-Tests pro Crate

Crate	Inhalt	bestanden	Fehler	ignoriert	Testzeit	Laufzeit cargo	Status
`cad-cli`	CLI	7	0	0	2,91 s	4,0 s	ok
`cad-kernel`	Kernel-Trait, Mesh, Export	0	0	0	0 s	0,2 s	ok
`cad-render`	Software-Rasterizer	4	0	0	1,50 s	2,4 s	ok
`cad-engine`	Auswertung, Cache, Provenienz, Zonen	31	0	0	6,99 s	8,0 s	ok
`cad-ir`	Geometriebaum, Cache-Schlüssel	0	0	0	0 s	0,2 s	ok
`cad-syntax`	Lexer, Parser, AST	16	0	0	0,00 s	0,2 s	ok
`cad-eval`	Sprachauswertung, Raster, Anker, Griffe	102	0	0	0,46 s	0,8 s	ok
`kernel-smlib`	SMLib-Fassade (nativ)	31	0	0	6,77 s	7,3 s	ok
`cad-session`	Sitzung, Protokoll	8	0	0	0,13 s	1,5 s	ok
`cad-wasm`	Wasm-Einstieg (Worker)	0	0	0	0 s	0,1 s	ok
`compat-tests`	OpenSCAD-Vergleich Stufe 1	3	0	0	0,04 s	0,2 s	ok
`kernel-mock`	Testkernel	0	0	0	0 s	0,1 s	ok
Summe		**202**	**0**	**0**	**18,80 s**	**25,2 s**	

Testzeit: Summe der von den Test-Binaries gemeldeten Zeiten (Debug-Build, SMLib und Fassade optimiert). Laufzeit cargo: `cargo test -p <crate>` nach vorherigem Build.

3.2 Wiederholungsläufe `kernel-smlib/tests/kernel.rs`

Modus	Läufe	fehlgeschlagen	Test	Dauer
seriell (`--test-threads=1`)	30	0	–	201 s
parallel (Standard, 4 Threads)	30	0	–	201 s
parallel (`--test-threads=8`)	30	0	–	203 s

Vor den Korrekturen: 4 von 40 Läufen mit 8 Threads fehlgeschlagen (zu dicker shell). Nach der rand()-Korrektur bei 100 Läufen mit 8 Threads noch je einmal `invalid_profiles_fail_cleanly` (Speicherfehler in SMLibs shell) und `handles_are_released` (paralleler Zähler); beides seither behoben.

3.3 Kernel als Wasm (Node)

`node crates/kernel-smlib/js/test-node.mjs`: 10 von 10 Tests bestanden (ok).

Test	Zeit	Status
box volume	23,1 ms	ok
box minus cylinder + provenance	88,1 ms	ok
fillet all edges of a box	330,8 ms	ok
fillet too large → error with edges	111,3 ms	ok
classify	1,9 ms	ok
tessellate	42,4 ms	ok
extrude, revolve, mirror	76,9 ms	ok
union of three orthogonal cylinders (checked boolean)	497,3 ms	ok
twisted extrude: rotate, then scale (OpenSCAD)	64,8 ms	ok
invalid handle	0,3 ms	ok

3.4 In diesem Durchgang behobene Fehler

Fehler	Ursache	Behebung, Nachweis
Union dreier orthogonaler Zylinder: 3237 statt 3800 mm³ (05_nested_booleans)	SMLib meldet bei (Z ∪ Y) ∪ X Erfolg, verwirft aber den −x-Arm des X-Zylinders (Hülle x ≥ −4); (Z ∪ X) ∪ Y ist richtig, (Y ∪ X) ∪ Z endet mit SM_ERR. Nur bei gleichen Radien (Schnittkurven mit Berührpunkten); mit r = 3,9 oder 4,1 für den dritten Zylinder stimmt es. `mass_props` und Mesh sind gleich falsch: kein Tessellierungs- oder Exportfehler. Paarweise rechnete die Fassade schon.	Die Fassade prüft jedes paarweise Ergebnis (Volumen-Ungleichungen, Lage von Proben je 5 pro Fläche dicht innen und außen an den Operandenflächen) und versucht sonst: Operanden tauschen, andere Reihenfolge, zuletzt zweiten Operanden um 10^{-6}…10^{-4} der Modellgröße verschieben; besteht nichts, bleibt SMLibs Ergebnis wie bisher. Tests: `kernel.rs` und `engine.rs::union_of_three_orthogonal_cylinders` (alle 6 Reihenfolgen, verschachtelt, als Bohrung), Node-Test.
linear_extrude mit twist: Hülle 1,25 mm daneben (12_linear_extrude_twist)	Schnitt t wurde erst skaliert, dann gedreht; OpenSCAD dreht erst und skaliert dann entlang x/y (`Scaling(s)·Rotation2D(-twist·t)`). Bei `scale = [1, 0.5]` entstehen andere Querschnitte (y bis 5,08 statt 3,83 mm). Drehsinn und Startwinkel waren richtig. Zusätzlich: Schnitte alle 22,5° (Hüllenfehler $3\cdot10^{-3}$ mm), und das Mesh lag 2,9 % über dem Volumen, weil SMLib die Sehnenhöhe nur an Dreieckskanten begrenzt.	Reihenfolge wie OpenSCAD; Schnitte alle 7,5° (Hülle auf 10^{-4} mm, Volumen 540,000 von 540). Die Tessellierung misst auf Freiformflächen die Abweichung der Dreiecksmitten und verfeinert bis ≤ 2·tol (höchstens 3 Durchgänge, Dreiecksbudget). Tests: `twisted_extrude_with_nonuniform_scale_matches_openscad`, `twisted_extrude_mesh_volume`, `engine.rs::linear_extrude_twist_nonuniform_scale_like_openscad`.

Wackliger Test `invalid_profiles_fail_cleanly`	Zwei Ursachen, beide im zu dicken `shell` (Würfel 10 mm, Wand 6 mm, muss scheitern). (1) SMLib ruft `srand(time(NULL))` in `SmFace::GetPointsInFace` und `SmShell::IsInnerShellOf` auf: vorher 4 von 40 Läufen (8 Threads) fehlgeschlagen, mit fester Uhrzeit (`time()` per LD_PRELOAD) 0 von 40. Ein `srand` vor jeder Operation reicht nicht, SMLib setzt den Startwert mitten in der Operation neu. (2) Danach noch 1 von 100: SMLibs Shell liest bei zu dicker Wand freigegebenen Speicher (valgrind: *Invalid read* in `SmProtoEdge::RemoveProtoVertex`; solidcore UB-239/242); je nach Heap scheitert sie, liefert einen Körper oder stürzt ab (Segfault in einer Schleife nach 9 Aufrufen).	(1) Die Fassade definiert `rand()` / `srand()` selbst (statisch gelinkt, nativ und Emscripten): fester Startwert, Neustart bei jedem `sc_*`-Aufruf. (2) `sc_shell` lehnt Wände ab, die zwischen zwei gegenüberliegenden ebenen Flächen keinen Platz haben (Abstand ≤ 2·t), bevor SMLib gerufen wird. Dazu lief `handles_are_released` parallel falsch (prozessweiter Zähler): eigenes Test-Binary. Abschnitt 3.2; Tests `determinism.rs`, `handles.rs`.
Diagonalbänder bei transparent gerenderten abgezogenen Körpern	`cad-render` sortierte transparente Dreiecke nach dem Schwerpunkt. Wo Dreiecke in der Tiefe überlappen, vor allem bei koplanaren Flächen (abgezogener Körper und sein Hohlraum im gedimmten Ergebnis), wechselt die Reihenfolge entlang der Dreiecksdiagonalen.	Fragmente werden pro Abtastpunkt nach Tiefe sortiert, koplanare in Szenenreihenfolge (Abb. *gehaeuse --line 12* in Abschnitt 5). Test: `render.rs::transparent_coplanar_faces_have_no_diagonal_bands`.

4 Vergleich mit OpenSCAD

`tests/openscad-compat/compare.py` exportiert jedes Modell als STL (`cad export`, Sehnentoleranz 0,01 mm) und vergleicht Volumen, Fläche und Hülle mit OpenSCAD 2021.01 (`$fn = 128` bei Rundungen, siehe `tests/openscad-compat/README.md`). Toleranzen: Hülle 0,01 mm; Volumen und Fläche 1 % (*polygonal*) bzw. 2 % (*rounded*). Ergebnis: **33 von 33** innerhalb der Toleranz, Laufzeit 11,3 s.

Modell	Klasse	V OpenSCAD [mm³]	V unseres [mm³]	ΔV	ΔA	ΔHülle [mm]	Dreiecke	Zeit	Status
`01_cube_variants`	polygonal	3 208,0	3 208,0	0,00 %	0,00 %	0,0000	60	0,05 s	ok
`02_sphere_variants`	rounded	1 016,9	1 013,7	0,31 %	0,15 %	0,0015	14 912	0,81 s	ok
`03_cylinder_variants`	rounded	2 104,0	2 101,5	0,12 %	0,05 %	0,0000	1 260	0,18 s	ok
`04_cone_variants`	rounded	1 342,0	1 340,8	0,09 %	0,05 %	0,0000	1 010	0,12 s	ok
`05_nested_booleans`	rounded	8 310,3	8 312,7	0,03 %	0,02 %	0,0000	6 688	1,52 s	ok
`06_hole_grid`	rounded	6 295,6	6 296,7	0,02 %	0,01 %	0,0000	6 252	1,46 s	ok
`07_crown_ring`	rounded	2 160,7	2 160,9	0,01 %	0,02 %	0,0000	1 088	0,83 s	ok
`08_module_children`	rounded	6 172,5	6 171,6	0,02 %	0,00 %	0,0000	2 072	0,44 s	ok
`09_rotate_mirror`	rounded	1 460,1	1 459,9	0,01 %	0,01 %	0,0000	324	0,07 s	ok
`10_scale_multmatrix`	polygonal	3 400,0	3 400,0	0,00 %	0,00 %	0,0000	48	0,05 s	ok
`11_linear_extrude_basic`	polygonal	1 513,0	1 513,0	0,00 %	0,00 %	0,0000	32	0,02 s	ok
`12_linear_extrude_twist`	rounded	2 688,9	2 660,9	1,04 %	1,49 %	0,0000	15 730	0,54 s	ok
`13_rotate_extrude_ring`	rounded	1 307,1	1 306,3	0,06 %	0,01 %	0,0000	17 120	0,66 s	ok
`14_pipe_bend`	rounded	1 573,9	1 572,4	0,09 %	0,03 %	0,0000	9 672	0,64 s	ok
`15_vase`	rounded	6 362,3	6 362,3	0,00 %	0,00 %	0,0000	2 556	0,20 s	ok
`16_polygon_holes`	polygonal	2 318,0	2 318,0	0,00 %	0,00 %	0,0000	80	0,04 s	ok
`17_polyhedron`	polygonal	626,7	626,7	0,00 %	0,00 %	0,0000	14	0,01 s	ok
`18_functions_let`	polygonal	7 040,0	7 040,0	0,00 %	0,00 %	0,0000	52	0,05 s	ok
`19_hex_nut`	rounded	264,7	264,9	0,07 %	0,01 %	0,0000	552	0,11 s	ok
`20_box_with_lid`	rounded	20 574,8	20 576,1	0,01 %	0,01 %	0,0000	2 284	0,76 s	ok
`21_angle_bracket`	rounded	10 408,0	10 408,3	0,00 %	0,00 %	0,0000	1 088	0,32 s	ok
`22_gear`	rounded	6 943,8	6 944,3	0,01 %	0,00 %	0,0000	1 616	0,37 s	ok
`23_shelf_holder`	rounded	10 423,8	10 424,1	0,00 %	0,00 %	0,0000	1 184	0,23 s	ok
`24_modifiers`	rounded	3 622,5	3 623,2	0,02 %	0,00 %	0,0000	284	0,06 s	ok
`25_root_modifier`	rounded	346,5	346,4	0,05 %	0,00 %	0,0000	268	0,06 s	ok
`26_if_else_variants`	rounded	1 661,6	1 660,5	0,07 %	0,04 %	0,0012	4 380	0,34 s	ok
`27_spacers_2d`	rounded	2 961,5	2 961,4	0,00 %	0,01 %	0,0000	2 512	0,47 s	ok

`28_cable_clip`	rounded	560,9	561,0	0,02 %	0,01 %	0,0013	796	0,20 s	ok
`29_assert_strings`	rounded	1 907,4	1 907,5	0,00 %	0,00 %	0,0000	276	0,08 s	ok
`30_each_ranges`	polygonal	832,0	832,0	0,00 %	0,00 %	0,0000	116	0,07 s	ok
`31_phone_stand`	polygonal	40 840,2	40 840,2	0,00 %	0,00 %	0,0000	84	0,03 s	ok
`32_low_fn_shapes`	polygonal	1 199,0	1 199,0	0,00 %	0,00 %	0,0000	102	0,04 s	ok
`33_global_fn`	rounded	1 625,6	1 627,4	0,11 %	0,04 %	0,0060	4 860	0,35 s	ok

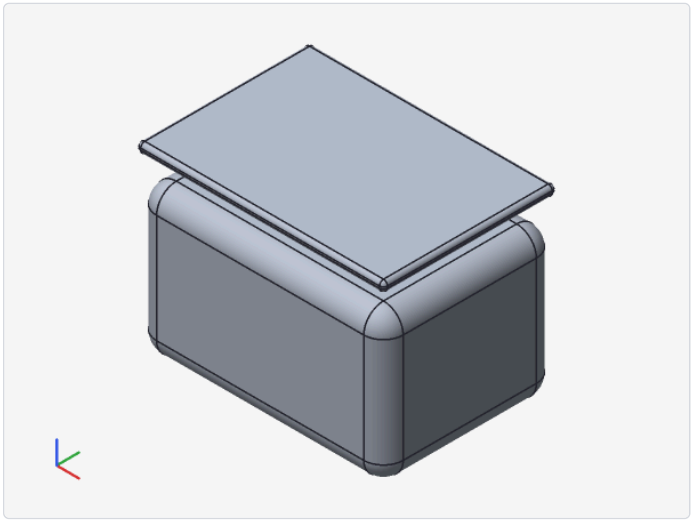
> **Zu 12_linear_extrude_twist:** Hier weicht OpenSCADs eigenes Netz vom exakten Körper ab. Exakt sind 1280 + 840 + 540 = 2660 mm³ (verdrillte Querschnitte behalten ihre Fläche); OpenSCAD liefert 2 688,9 mm³, weil es jede verdrillte Seitenfläche in zwei ebene Dreiecke mit fester Diagonale teilt. Unser STL hat 2 660,9 mm³. Die Abweichung von 1,04 % in der Tabelle ist also OpenSCADs Facettierung.

5 Renderings

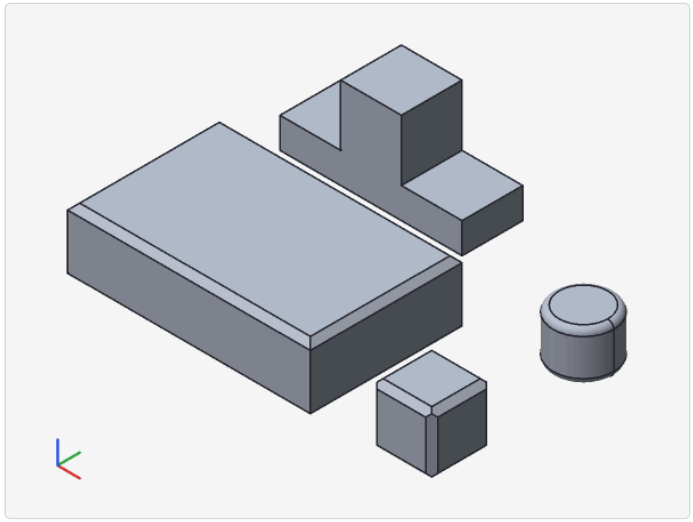
Alle Bilder 600×450 px, isometrische Ansicht (Blick entlang (−1, 1, −1), orthografisch), `cad render <datei> --size 600x450`. OpenSCAD-Bilder: `openscad -o x.png --render --imgsize=600,450 --viewall --autocenter --projection=ortho --camera=100,-100,100,0,0,0` mit denselben `-D`-Werten wie die Referenz.

5.1 Beispiele der Spec (`examples/spec/`)

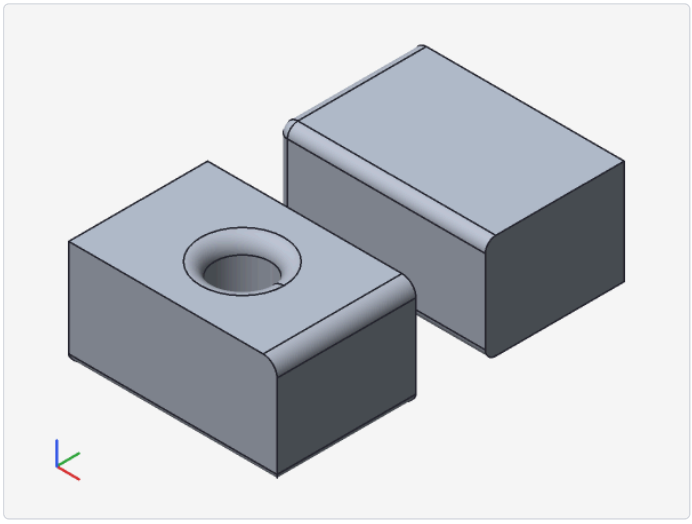
Nur mit unserer CLI: Raster, Anker, Fillets und Zonen gibt es in OpenSCAD nicht.



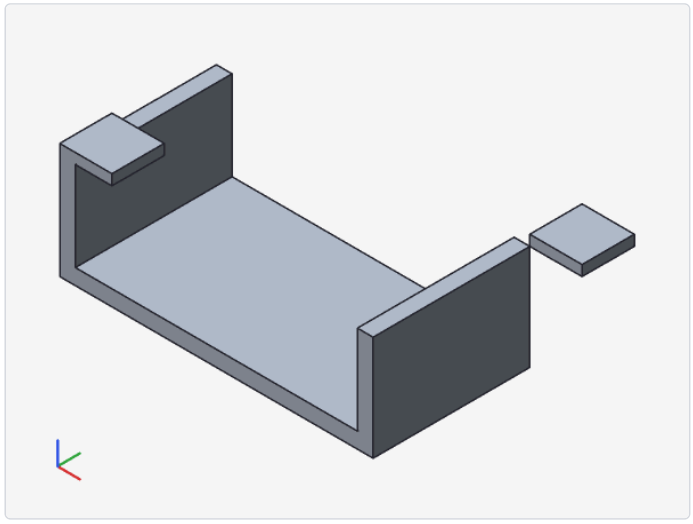
dose_shell – shell(t) { körper; zone } – Dose aus einem verrundeten Vollkörper ($fillet), oben geöffnet über die Zone an der Rasterebene deckel; Anker für den Deckel.



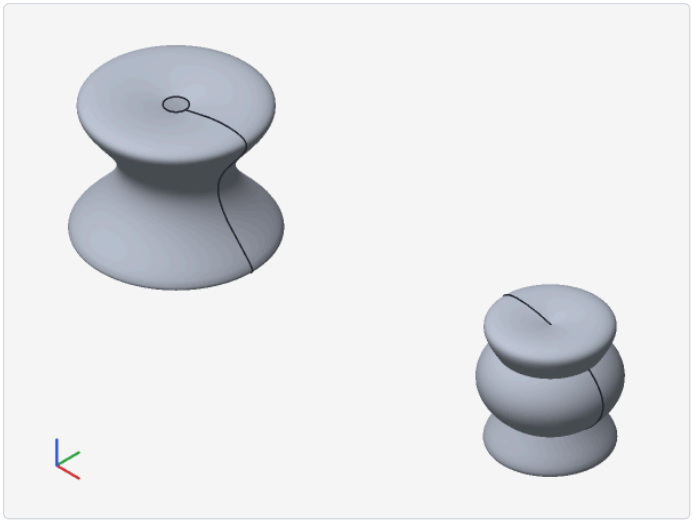
fase_chamfer – Fasen in allen drei Formen – chamfer-Block mit Zone, union(chamfer = ...) und $chamfer auf Primitiven; dazu $fillet für einen verrundeten Knopf.



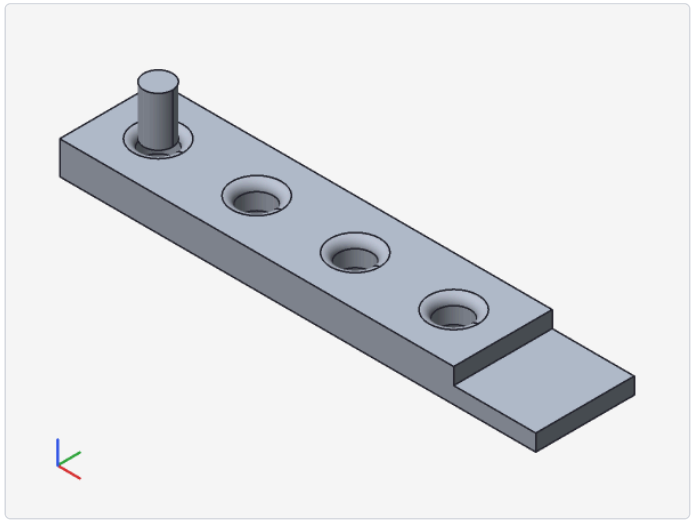
fillet_zonen – Zonenblock fillet(r) { körper; zone; ... } mit Rasterlinie, Rasterebene und Volumenzone, select = "touch", verschachtelte Blöcke mit zwei Radien.



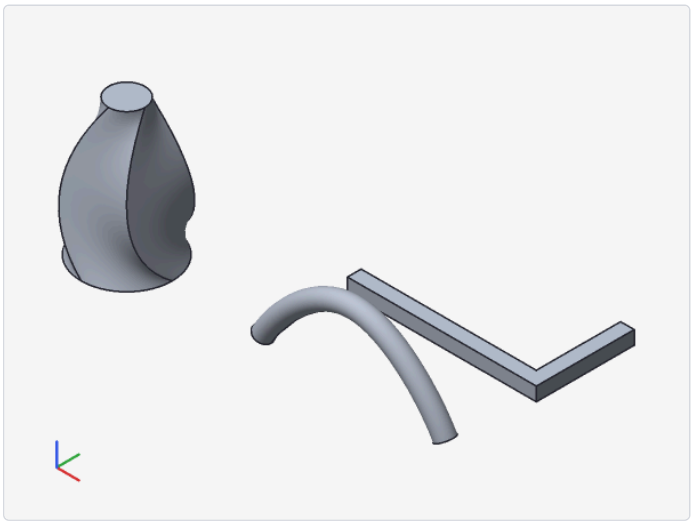
gehaeuse – Spec-Beispiel „gehaeuse“ – Raster g im Modul, Box zwischen Rasterpunkten, Anker nach außen, benannte Instanz k, Deckel am Anker und ein zweites Raster aus k.ecke.x.



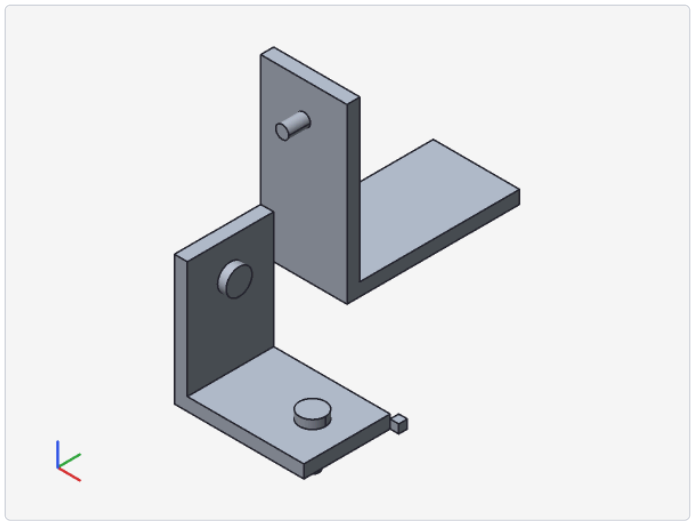
knauf_mirror – Knauf – 2D-Raster für ein Splineprofil, rotate_extrude, torus, difference(fillet = ...) an der Bohrung; gespiegelte Kopie an einer Ebene eines skalierten Rasters.



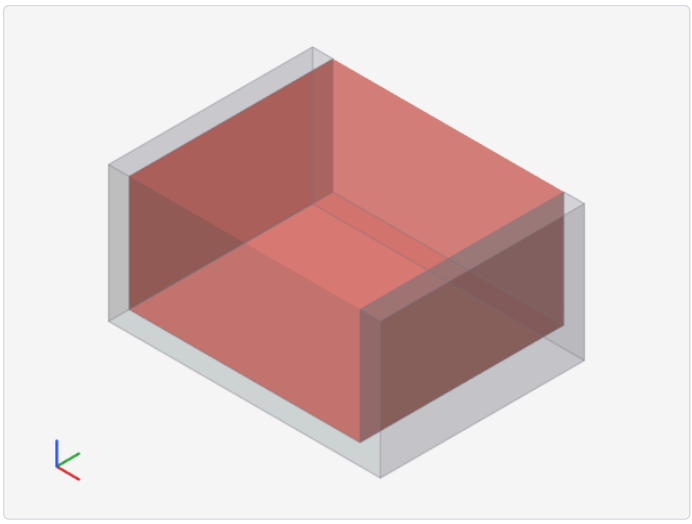
lochplatte – Lochplatte mit indizierten Rasterebenen (Listen), Bohrungen an Rasterpunkten, an Rasterpunkten platziertes zweites Raster und difference(fillet = 1) für die Bohrkanten.



loft_sweep – Profiloperationen – loft vom Kreis zum Quadrat (Trichter), sweep eines Kreises entlang eines Spline-Pfads (Griffbügel) und eines linearen Pfads aus Rasterpunkten.



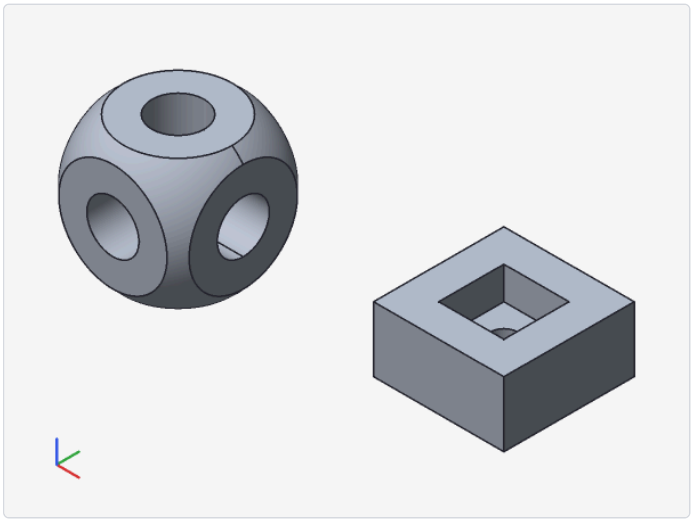
winkel_anker – Anker von außen – zwei benannte Instanzen eines Winkels, Schrauben an den Ankern mit rotate(to = dir(k.a)), gedrehte Instanz, Raster aus Ankerkoordinaten.



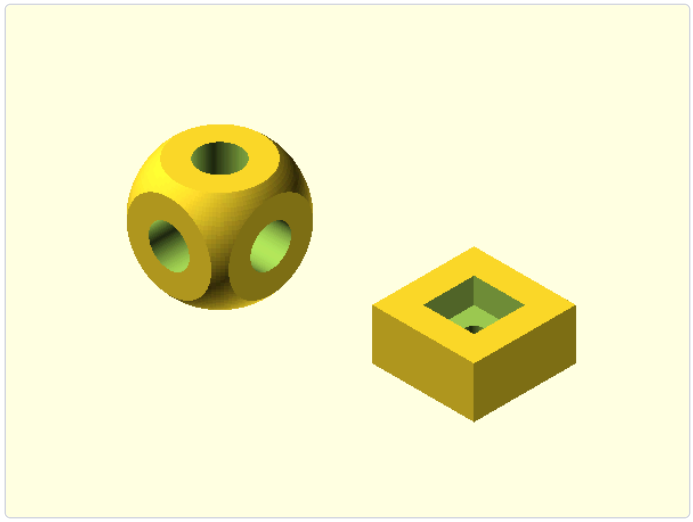
gehaeuse --line 12 – Zeile 12: abgezogener Körper rot transparent, Rest gedimmt

5.2 OpenSCAD-Modelle: unseres (links) und OpenSCAD (rechts)

`05_nested_booleans` verschachtelte Booleans – Würfel mit Kugel geschnitten, Kreuzbohrung abgezogen.

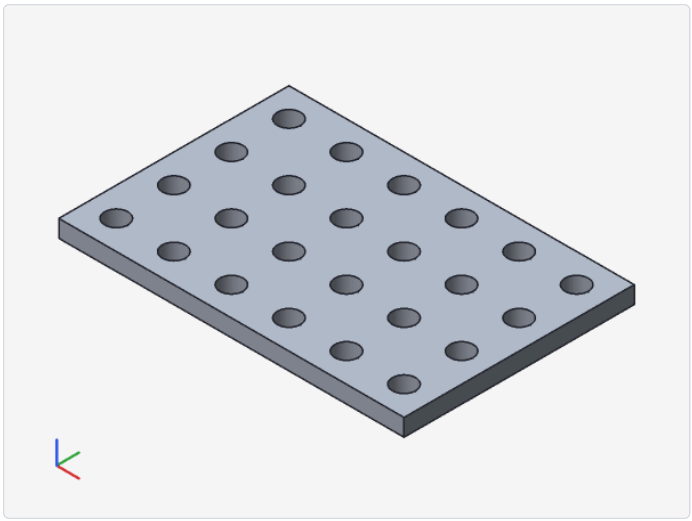


cad render

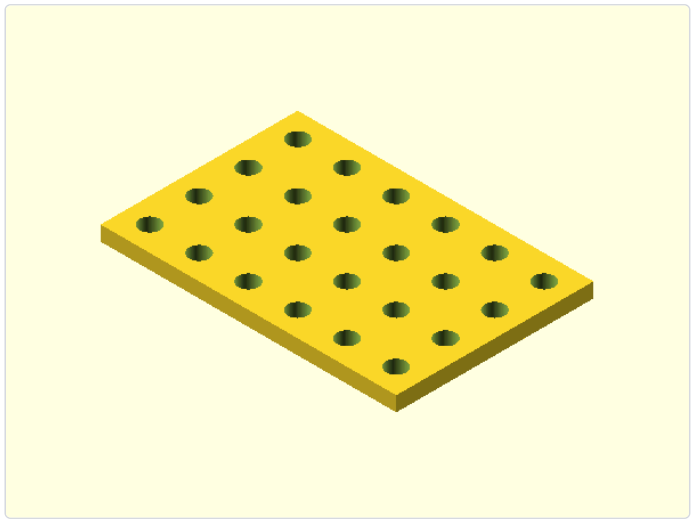


OpenSCAD 2021.01

`06_hole_grid` Lochrasterplatte – eine for-Schleife mit zwei Variablen erzeugt ein Bohrungsraster.

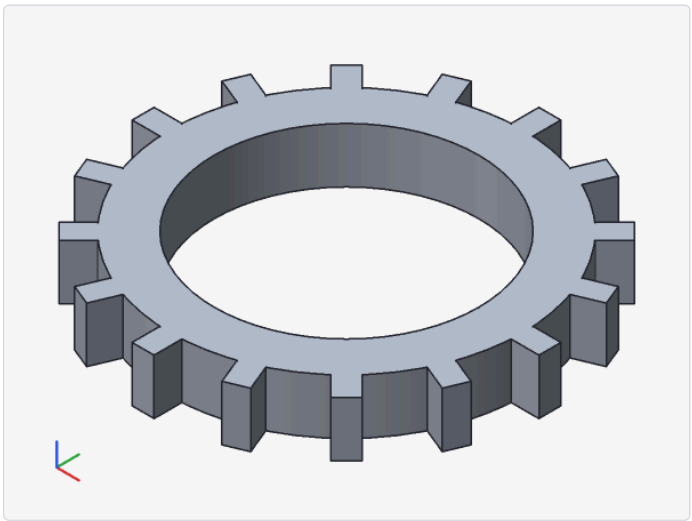


cad render

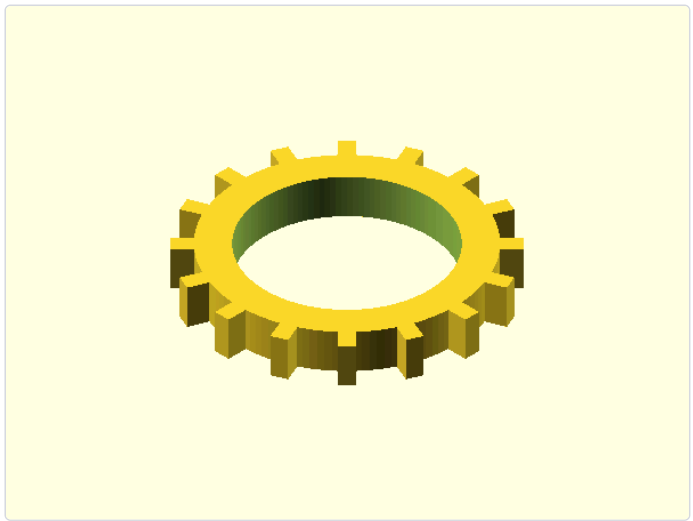


OpenSCAD 2021.01

`07_crown_ring` Zahnkranz-artiger Ring – Zähne per for und rotate gleichmäßig verteilt.

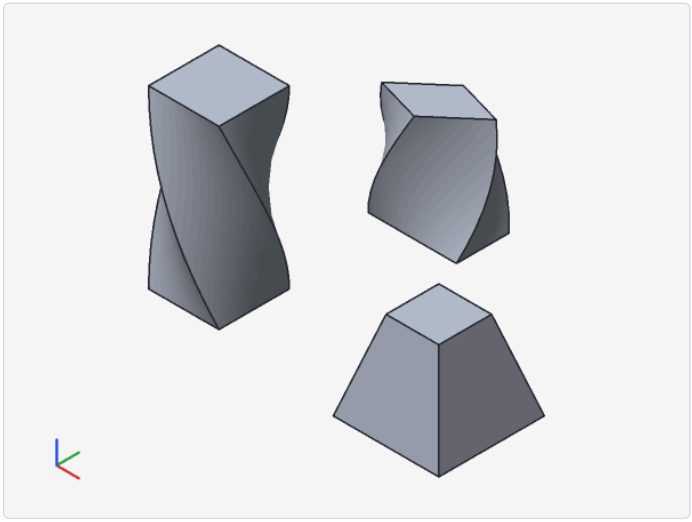


cad render

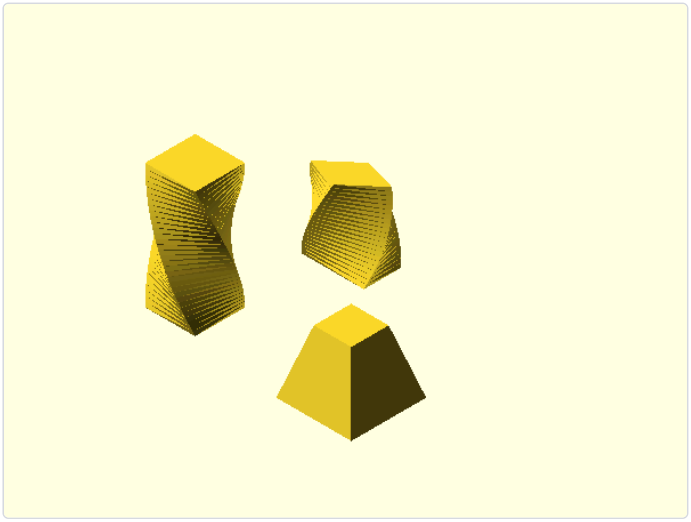


OpenSCAD 2021.01

`12_linear_extrude_twist` linear_extrude mit twist und scale – gedrehter Stab und verjüngter Sockel.

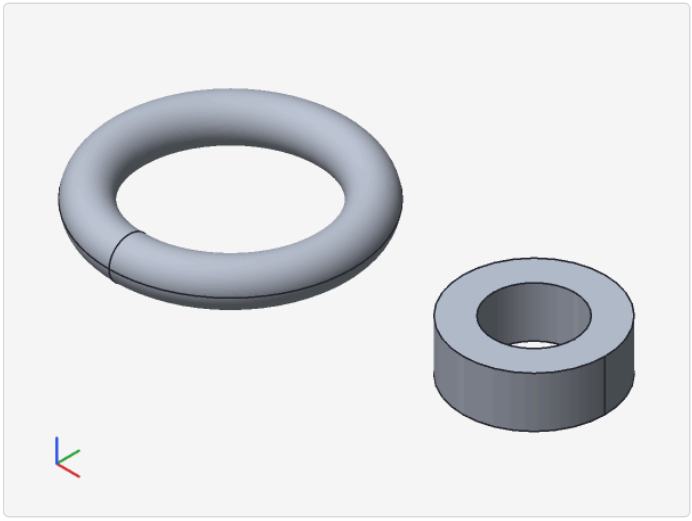


cad render

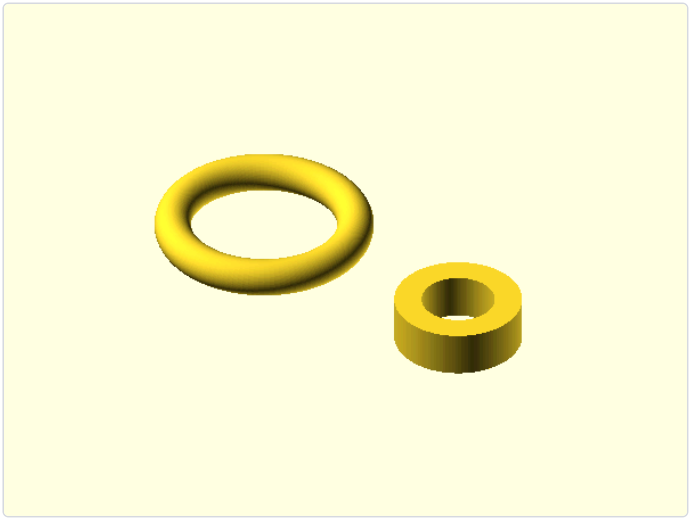


OpenSCAD 2021.01

`13_rotate_extrude_ring` rotate_extrude eines verschobenen Kreises (Ring) und eines Rechtecks (Scheibe mit Loch).



cad render

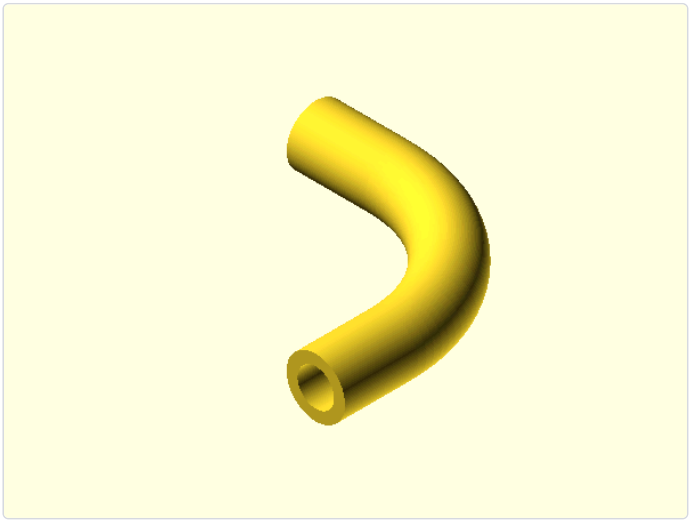


OpenSCAD 2021.01

`14_pipe_bend` Rohrbogen per rotate_extrude(angle = 90) mit zwei geraden Anschlussstücken.

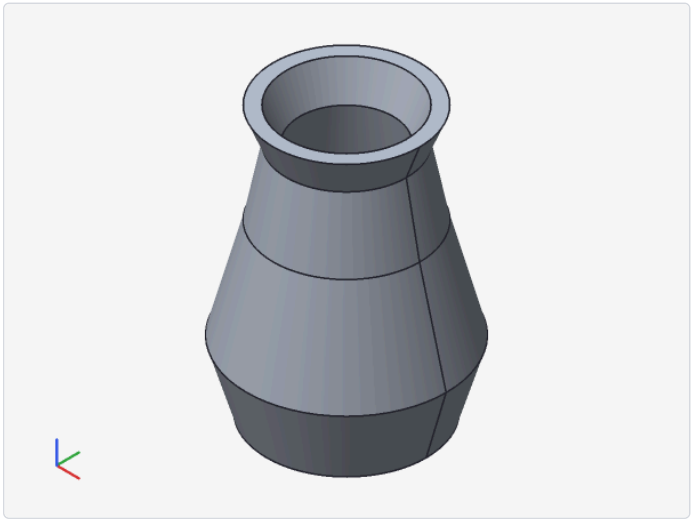


cad render

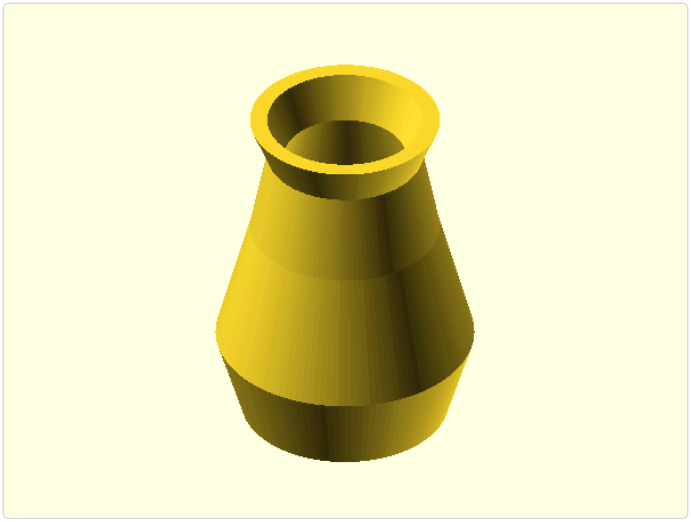


OpenSCAD 2021.01

`15_vase` **Vase als rotate_extrude eines Polygonprofils (Wand 2 mm, Boden 2 mm).**

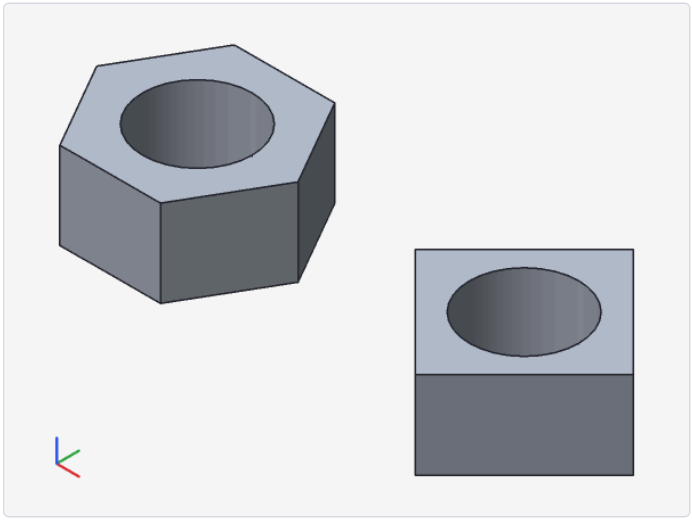


cad render

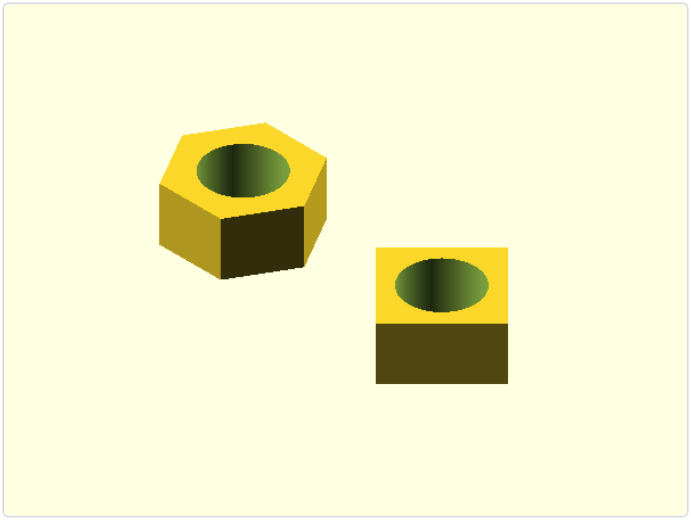


OpenSCAD 2021.01

`19_hex_nut` **Sechskantmutter M5 – $fn = 6 am Aufruf erzeugt das Sechseck, die Bohrung ist rund.**

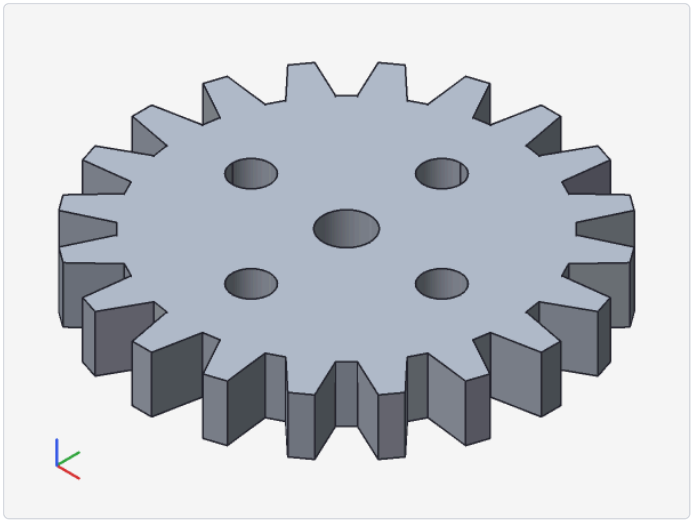


cad render

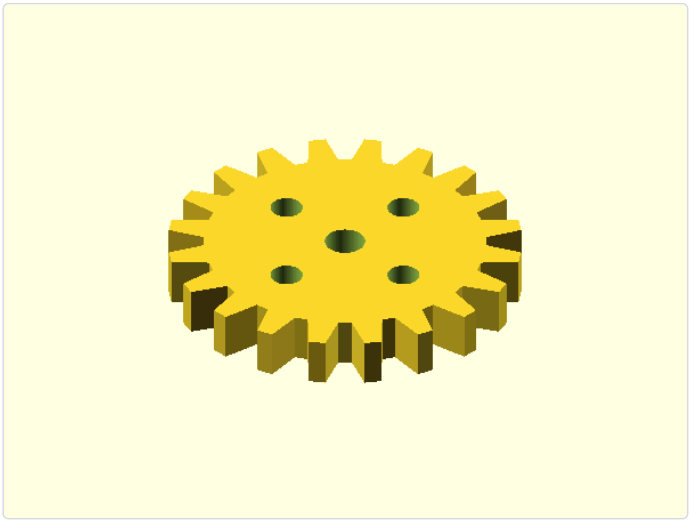


OpenSCAD 2021.01

`22_gear` **vereinfachtes Stirnrad – Trapezzähne als polygon aus einer Listen-Comprehension.**

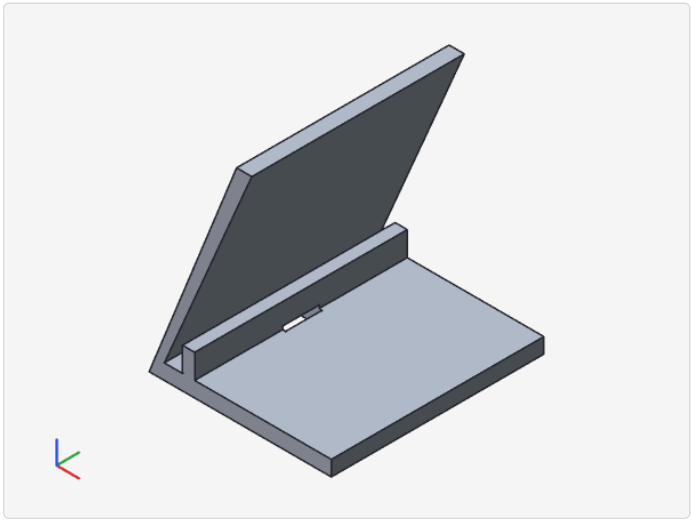


cad render

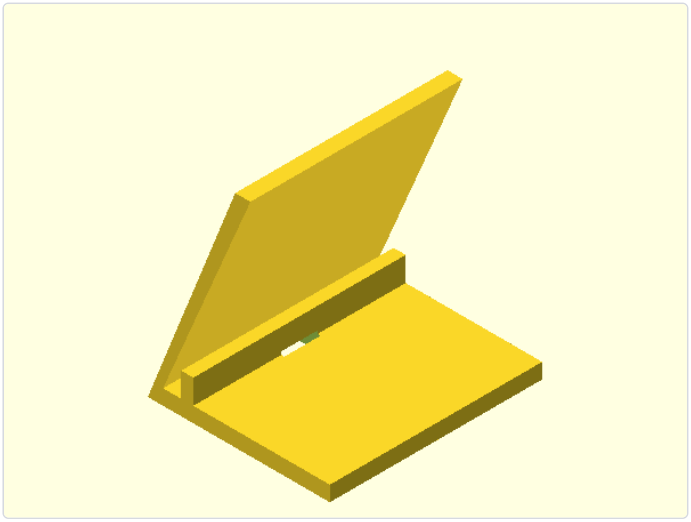


OpenSCAD 2021.01

`31_phone_stand` **Handyständer – geneigte Lehne über rotate, Lippe, Kabelschlitz; Profil per polygon.**



cad render



OpenSCAD 2021.01

6 Messwerte

6.1 Wasm-Größen

Datei	roh	gzip −9	Brotli q11
`smlib.wasm`	4 993 KiB	1 769 KiB	1 270 KiB
`smlib.js`	14 KiB	5 KiB	4 KiB
`cad_wasm_bg.wasm`	1 186 KiB	464 KiB	353 KiB
`cad_wasm.js`	28 KiB	6 KiB	5 KiB

`smlib.wasm`: SMLib + SM_API + Fassade, Emscripten `-O3 -fwasm-exceptions`; `cad_wasm_bg.wasm`: Sprache, Engine, Sitzung (wasm-bindgen, wasm-opt −O3).

6.2 Ladezeit und Auswertung (Node 22, ein Prozess wie im Worker)

`web/scripts/smoke-node.mjs --json` mit dem Beispiel `gehaeuse`, Median aus 3 Läufen, Schrittbudget 16 ms.

Schritt	Zeit
smlib.wasm laden, kompilieren, initialisieren	32,4 ms
cad_wasm laden und initialisieren	32,4 ms
SetSource → vorläufige Szene (ohne Kernel)	20,7 ms
SetSource → vollständige Szene	123,1 ms
SetSource → Wurzel-Mesh	141,3 ms
Meshes aller Knoten anfordern	6,1 ms
Griff ziehen → TextEdit → neue Szene	12 ms
… → grobes Wurzel-Mesh	14,1 ms
erneutes SetSource (Cache warm) → vorläufige Szene	0,9 ms
erneutes SetSource (Cache warm) → Szene	21,9 ms
erneutes SetSource (Cache warm) → Wurzel-Mesh	23,9 ms
Auswertschritte bis zur vollständigen Szene	5
Dreiecke Wurzel-Mesh / Knoten der Szene	40 / 8

Nur Kernel (`test-node.mjs`): Datei lesen 4,5 ms, kompilieren, instanziieren und initialisieren 29,0 ms. Szenario: box-cylinder + fillet(5 top edges) + tessellate (1376 triangles): primitives 4.1 ms, boolean 24.4 ms, edges 0.5 ms, fillet 64.3 ms, tessellate 44.3 ms; total incl. load 1410 ms.

6.3 Auswertung nativ (CLI)

`cad export` je Modell (Prozessstart, Auswertung, Tessellierung, STL): Median 0,20 s, Summe 11,2 s; am längsten `05_nested_booleans` 1,52 s, `06_hole_grid` 1,46 s, `07_crown_ring` 0,83 s.

`cad render` der Spec-Beispiele (600×450, 2×2-Überabtastung): dose_shell 0,69 s, fase_chamfer 0,26 s, fillet_zonen 0,31 s, gehaeuse 0,06 s, knauf_mirror 2,67 s, lochplatte 0,51 s, loft_sweep 0,47 s, winkel_anker 0,46 s.

7 Bekannte Grenzen

- SMLib-Booleans sind in Sonderlagen unzuverlässig. Die Fassade erkennt falsche Ergebnisse über Stichproben, nicht vollständig: Ein Fehler zwischen allen Probepunkten bleibt unentdeckt. Die Prüfung nutzt SMLibs Punktklassifizierung, die bei gedrehten Splinekörpern selbst irrt (knauf_mirror); besteht keine Variante, bleibt deshalb SMLibs eigenes Ergebnis. Im Verschiebe-Rückfall weicht der zweite Operand um bis zu 10^{-4} der Modellgröße ab. Die

Prüfung kostet Zeit: `cad export` aller 41 Beispiele beim Beheben gemessen 13,7 s → 18,9 s (+38 %), am meisten bei Vereinigungen vieler Teile (07_crown_ring ×2,1) und bei knauf_mirror (×1,9, Rückfälle).

- Offene Ergebnisse: Berühren sich Körper an einer Kante, ist die Vereinigung in SMLib kein geschlossener Körper; `cad` meldet dann Volumen 0 (winkel_anker; das STL ist in Ordnung). knauf_mirror: Der gedrehte Splinekörper minus Bohrung ist offen, Torus und Stift fehlen im Ergebnis, das Fillet an der Bohrung scheitert (SM_ERR) – alles wie vor diesem Durchgang, noch offen.
- Tessellierung von Freiformflächen: Die Verfeinerung setzt eine globale maximale Kantenlänge, damit werden auch ebene Flächen desselben Körpers feiner (12_linear_extrude_twist: 15 730 Dreiecke, OpenSCAD 452).
- Twist-Extrusion ist eine kubische Loft-Annäherung durch Schnitte alle 7,5°. Kreisprofile mit ungleichmäßigem `scale` und `scale = 0` (Spitze) werden nicht unterstützt.
- OpenSCAD facettiert, wir rechnen exakt: Volumen runder Teile weichen systematisch um 0,01–0,3 % ab; bei twist liegt OpenSCAD selbst um ~1 % daneben (Abschnitt 4).
- Determinismus: `rand()` ist fest und prozessweit ersetzt. Speicherfehler in SMLib machen Ergebnisse vom Heap abhängig: Die Shell-Vorprüfung deckt nur ebene, gegenüberliegende Wände ab; schon knapp passende Wände (10-mm-Würfel, t = 4,99) zeigen unter valgrind ungültige Lesezugriffe. Abhilfe bringen erst die Upstream-Korrekturen (tangency-labs/smlib 93efd6d, af8db37) bzw. solidcore.
- `cad render` ist ein Software-Rasterizer (orthografisch, Lambert, 2×2-Überabtastung) ohne Schatten und Perspektive. Transparenz pro Pixel braucht Speicher proportional zur Zahl der Schichten.
- Nicht Teil dieses Berichts: Browser-UI, STEP-Export, Umstieg auf solidcore (M8).

Erzeugt mit `python3 tools/report/report1.py` (PDF: `tools/report/print-pdf.mjs`, Chromium aus Playwright).